1. We use a link for the shortest path computation only if it exists in both directions because communication needs to be bidirectional; in order to receive a response, there must be a route from the destination to the source.

2. The algorithm does not necessarily produce symmetrical routes. If there are only bidirectional links and the link costs are symmetrical, and there is a unique shortest path, then the produced routes will be symmetrical. But if there are multiple shortest paths, then the algorithm will return whichever it finds first, which may be different depending on direction. Similarly, if weights between graph edges are not symmetrical, the weight for a given path in one direction might not be the same in the other direction, which can result in different paths being the shortest depending on direction.

3. If a node advertises itself as having neighbors, but never forwards packets, then any route that traverses it will have its packets lost at that node. One way to deal with this might be to validate forwarding, not just adjacency. So when forwarding a packet to a given node, listen to hear if it forwards the packet. If it fails too often, then we can treat the node like it doesn't exist.

4. If link state packets are lost or corrupted, it can result in stale or inconsistent topology views. Nodes would keep an older or corrupted view of the network topology. Packets might get forwarded along suboptimal paths or choose a next hop that no longer exists.

5. If a node repeatedly alternated between advertising and withdrawing a neighbor, it would result in a lot of wasted CPU time recomputing the network topology. The first solution that comes to mind is to debounce the neighbor discovery computation, and potentially temporarily blacklist nodes that repeatedly send updates during the debounce period.